

TP2 - Programmation Orientée Objet en TypeScript

Exercice 1 :

Partons d'une classe `Vehicle`, celle ci possède 3 attributs privés :

- `fuel`: string
- `fuelLevel`: number
- `speed`: number

Le constructeur reçoit ces paramètres dans l'ordre indiqué.
Créer les getters pour ses attributs.

Exercice 1.1 :

Ajouter à la classe `Vehicle` la méthode `toString`: string, qui retourne les infos du véhicule.

Ajouter à la classe `Vehicle` la méthode `move`: boolean.

Cette méthode ne prend pas de paramètres. Si le carburant est supérieur ou égal à 5, baisser de 5 la valeur de `fuelLevel` et retourner vrai, sinon retourner faux.

Ajouter également également un attribut `distanceTravelled`: number, qui est initialisé à 0 dans le constructeur.

A chaque appel de la fonction `move`, si le déplacement est possible, ajouter l'attribut `speed` à l'attribut `distanceTravelled`.

Exercice 2 :

Créer une classe `Car`, celle-ci devra hériter de la classe `Vehicle`.

Elle aura également 1 attribut supplémentaire :

- `carBrand`: string

Surcharger la méthode `toString` de `Vehicle`.

Noter aussi ce qu'il se passe lorsqu'on essaie d'accéder à des attributs de la classe mère.

Mettre les attributs de la classe `Vehicle` en protégés.

Exercice 2.1 :

Ajouter à la classe `Car` la méthode `fillFuel`: void, qui prend en paramètre un nom de carburant.

Vérifier que le type de carburant passé est le bon, celui-ci doit être égal à "gasoline". Si c'est le cas, `fuelLevel` doit être mis à la valeur 100. Si ce n'est pas le cas, vous devez lever une exception.

Exercice 2.2 :

Ajouter à la classe `Vehicle` une méthode `klaxon` : `void`, qui affiche le bruit du véhicule.

Pour la classe `Vehicle`, cela lèvera une erreur.

Créer la même méthode sur la classe `Car`, cette fois l'appel de la méthode affichera "Tuut"

Exercice 3 :

Créer une classe `Donkey`, celle-ci sera identique à `Car`, à la différence que :

- `fillFuel` prendra un carburant nommé « carrot », qui ajoutera 25 au carburant (maximum 100)
- La méthode `klaxon` affichera "Hii Han"

Exercice 3.1 :

Pour illustrer la notion de polymorphisme, nous allons créer un tableau de `Vehicle` et y mettre des objets de type `Car` et `Donkey`. Il est très important de comprendre pourquoi TypeScript nous permet d'instancier des objets autres que `Vehicle` dans ce contexte.

Boucler sur le tableau pour afficher l'ensemble des objets.

Exercice 4 :

Passer la classe `Vehicle` en classe abstraite. Noter alors ce qu'il se passe lorsque l'on essaye d'instancier un objet depuis la classe `Vehicle`.

Définir la méthode `fillFuel` dans la classe `Vehicle` comme abstraite. Regarder alors ce qu'il se produit lorsqu'une de ses classes filles ne la surcharge pas.

Exercice 5 :

Créer une interface `Immatriculation` dans le dossier `interfaces`. Celle-ci implémente un attribut public `immatriculation`.

Implémenter l'interface `Immatriculation` sur la classe `Car`.

Exercice 5.1 :

Créer une interface `Refuelable` que pourront implémenter `Car` et `Donkey` avec la déclaration de `fillFuel` avec leur type de carburant.

Exercice 6.1 :

Créer un système où chaque instance d'un véhicule créé à un id unique (mixer un attribut d'instance et un attribut statique).

Exercice 6.1 :

Créer un tableau d'objets `Vehicle` avec 6 entrées.

Simuler une course automobile. Les voitures vont rouler jusqu'à ne plus avoir de carburant.

Afficher alors le classement des voitures triées par distance parcourue.